LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK

LAPORAN AKHIR PRAKTIKUM PEKAN 1-5

Disusun Oleh:

Muhammad Aufa Rafiki

2511531012

Dosen Pengampu: AFDHAL DINILHAK, M.Kom

Asisten Praktikum: Ezza Addini



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

LAPORAN PRATIKUM
PEMOGRAMAN BERORIENTASI OBJEK

LAPORAN PRAKTIKUM PEKAN 1

Disusun Oleh:
Muhammad Aufa Rafiki

2511531012

Dosen Pengampu: Nurfiah, S.ST, M.Kom

Asisten Praktikum: Ezza Addini



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

PEMBAHASAN

A. Langkah Kerja Praktikum

1. Membuat Class Rekening
   File bernama Rekening.java yang berfungsi sebagai model/blueprint utama.
   Langkah-langkah:
   1) Buat class bernama Rekening

```
3  public class Rekening {
4
5  }
```

   2) Tambahkan atribut variabel ke dalam class Rekening untuk menyimpan data setiap akun:
      - nomorRekening (tipe data String)
      - namaPemilik (tipe data String)
      - saldo (tipe data double)

```
String nomorRekening;
String namaPemilik;
double saldo;
```

   3) Buat constructor pada class Rekening untuk menginisialisasi atribut saat objek baru dibuat menggunakan keyword new. Constructor ini menerima parameter nomor, nama, dan saldo awal.

```
public Rekening(String nomor, String nama, double saldoAwal) {
    nomorRekening = nomor;
    namaPemilik = nama;
    saldo = saldoAwal;
    System.out.println("Rekening atas nama " + namaPemilik + "berhasil dibuat dengan saldo Rp" + saldo);
}
```

   4) Tambahkan method untuk melakukan manipulasi state saldo dan menampilkan informasi akun:
      - Method setorTunai(double nominal): Memvalidasi nominal setor harus lebih dari 0. Jika valid, saldo akan bertambah. Jika tidak, maka akan menampilkan pesan gagal.
      - Method cekInformasi(): Menampilkan rincian data rekening ke konsol.

```
public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
    } else {
        System.out.println("Gagal: Nominal setor harus lebih dari 0!");
    }

}

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir  : Rp" + saldo);
    System.out.println("----------------------");
}
```

   5) Maka isi class Rekening yang lengkap sebagai berikut:

```java
package Pekan1_2511531012;

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    public Rekening(String nomor, String nama, double saldoAwal) {
        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;
        System.out.println("Rekening atas nama " + namaPemilik + "berhasil dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }

    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir  : Rp" + saldo);
        System.out.println("----------------------");
    }

}
```

2. Membuat Engine CLI pada Main.java
   Class Main dibuat untuk mengontrol alur aplikasi interaktif berbasis teks (CLI) menggunakan input dari pengguna.
   Langkah kerja:
   1) Buat file baru bernama Main.java dan import library Scanner untuk membaca input dari keyboard

```java
package Pekan1_2511531012;

import java.util.Scanner;

public class Main {
    public static void main(String[]args) {
```

   2) Buat Deklarasi Variabel
      Di dalam method main, persiapkan kontrol program yang terdiri dari inisialisasi objek Scanner, deklarasi penampung objek akunAktif, dan inisialisasi variabel kontrol isRunning
      - Inisialisasi Objek Scanner (Scanner input = new Scanner(System.in)): Membuat objek Scanner untuk membaca input pengguna dari keyboard.
      - Deklarasi Variabel Referensi (Rekening akunAktif = null): Deklarasi variabel penampung objek bernilai null.
      - Inisialisasi Variabel Kontrol (boolean isRunning = true): Menyiapkan variabel tipe boolean untuk mengontrol perulangan menu while

```java
public static void main(String[]args) {
Scanner input = new Scanner(System.in);
Rekening akunAktif = null;
boolean isRunning = true;
```

   3) Buat Perulangan Menu

Membuat perulangan while (isRunning) yang di dalamnya menampilkan daftar menu teks dan membaca pilihan angka dari pengguna. Menggunakan input.nextInt() untuk mengambil pilihan menu berupa angka, diikuti oleh input.nextLine() untuk membersihkan karakter enter (buffer) agar input teks berikutnya tidak tertembus/terlewati secara otomatis.

```
while (isRunning) {
    System.out.println("\nMenu Utama:");
    System.out.println("1. Buka Rekening Baru");
    System.out.println("2. Setor Tunai");
    System.out.println("3. Tarik Tunai");
    System.out.println("4. Cek Informasi Rekening");
    System.out.println("5. Ganti Akun");
    System.out.println("0. Keluar");
    System.out.println("Pilih menu:");

    int pilihan = input.nextInt();
    input.nextLine();
```

4) Tambahkan kontrol logika menu menggunakan switch-case untuk mengarahkan eksekusi program berdasarkan angka yang dipilih pengguna:
   - Case 1 (Buka Rekening): Mengambil input nomor, nama, dan saldo awal dari user, lalu menginstansiasi objek Rekening baru.
   - Case 2 (Setor Tunai): Memeriksa apakah akunAktif sudah diinisialisasi. Jika ada, panggil method setorTunai().
   - Case 3 (Tarik Tunai): Menampilkan pesan konfirmasi penanganan tugas mandiri.
   - Case 4 (Cek Informasi): Memanggil method cekInformasi().
   - Case 0 (Keluar): Menghentikan loop program

```
    switch (pilihan) {
        case 1:
            System.out.print("Masukkan No Rekening: ");
            String no = input.nextLine();
            System.out.print("Masukkan Nama Pemilik: ");
            String nama = input.nextLine();
            System.out.print("Masukkan Saldo Awal: ");
            Double saldo = input.nextDouble();
            akunAktif = new Rekening(no, nama, saldo);
            break;

        case 2:
            if (akunAktif == null) {
                System.out.println("Error: mohon maaf, anda belum memiliki nomor rekening!");
            } else {
                System.out.println("Masukkan nominal setor: ");
                double setor = input.nextDouble();
                akunAktif.setorTunai(setor);
            }
            break;

        case 3:
            System.out.println("Fitur ini akan dikerjakan sebagai Tugas Mandiri.");
            break;

        case 4:
            if (akunAktif == null) {
                System.out.println("Error: Anda belum membuka rekening!");
            } else {
                akunAktif.cekInformasi();
            }
            break;

        case 0:
            isRunning = false;
            System.out.println("Sistem ditutup. terima kasih!");
            break;

        default:
            System.out.println("Pilihan tidak valid!");
    }
```

5) Kode Program Lengkap:

```java
package Pekan1_2511531012;

import java.util.Scanner;

public class Main {
    public static void main(String[]args) {
    Scanner input = new Scanner(System.in);
    Rekening akunAktif = null;

    boolean isRunning = true;
    System.out.println("\"--- SISTEM PERBANKAN MINI ---\"");
    while (isRunning) {
        System.out.println("\nMenu Utama:");
        System.out.println("1. Buka Rekening Baru");
        System.out.println("2. Setor Tunai");
        System.out.println("3. Tarik Tunai");
        System.out.println("4. Cek Informasi Rekening");
        System.out.println("5. Ganti Akun");
        System.out.println("0. Keluar");
        System.out.println("Pilih menu:");

        int pilihan = input.nextInt();
        input.nextLine();

        switch (pilihan) {
            case 1:
                System.out.print("Masukkan No Rekening: ");
                String no = input.nextLine();
                System.out.print("Masukkan Nama Pemilik: ");
                String nama = input.nextLine();
                System.out.print("Masukkan Saldo Awal: ");
                Double saldo = input.nextDouble();
                akunAktif = new Rekening(no, nama, saldo);
                break;

            case 2:
                if (akunAktif == null) {
                    System.out.println("Error: mohon maaf, abda belum memiliki nomor rekening!");
                } else {
                    System.out.println("Masukkan nominal setor: ");
                    double setor = input.nextDouble();
                    akunAktif.setorTunai(setor);
                }
                break;

            case 3:
                System.out.println("Fitur ini akan dikerjakan sebagai tugas Mandiri.");
                break;

            case 4:
                if (akunAktif == null) {
                    System.out.println("Error: Anda belum membuka rekening!");
                } else {
                    akunAktif.cekInformasi();
                ]
                break;

            case 0:
                isRunning = false;
                System.out.println("Sistem ditutup. Terima kasih!");
                break;

            default:
                System.out.println("Pilihan tidak valid!");

        }
    }
    input.close();
    ]
}
```

B. Tugas/Latihan Praktikum

1. Implementasi Fitur Tarik Tunai
   - Tambahkan method tarikTunai(double nominal) ke dalam class Rekening.
   - Aturan : Saldo tidak boleh menjadi minus dan minimal nominal Tarik tunai adalah 10.000. Jika pengguna mencoba menarik nominal yang lebih besar dari saldo saat ini atau nominal kecil dari 10.000, program tidak boleh crash/error, melainkan menampilkan peringatan ke konsol: "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp[nominal]" atau “Transaksi Gagal : Minimal nomonal penarikan 10.000”.
   - Terapkan method ini dengan menu nomor 3 pada class Main.

Kode Program:

Kode program pada class Rekening:

```
32
33●     public void tarikTunai(double nominal) {
34●         if (nominal < 10000){
35              System.out.println("Gagal: Nominal tarik tunai harus lebih dari 10000!");
36●         } else if (nominal > saldo) {
37              System.out.println( "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
38●         } else {
39              saldo -= nominal;
40              System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
41          }
42      }
43
44  }
```

Kode Program pada class Main:

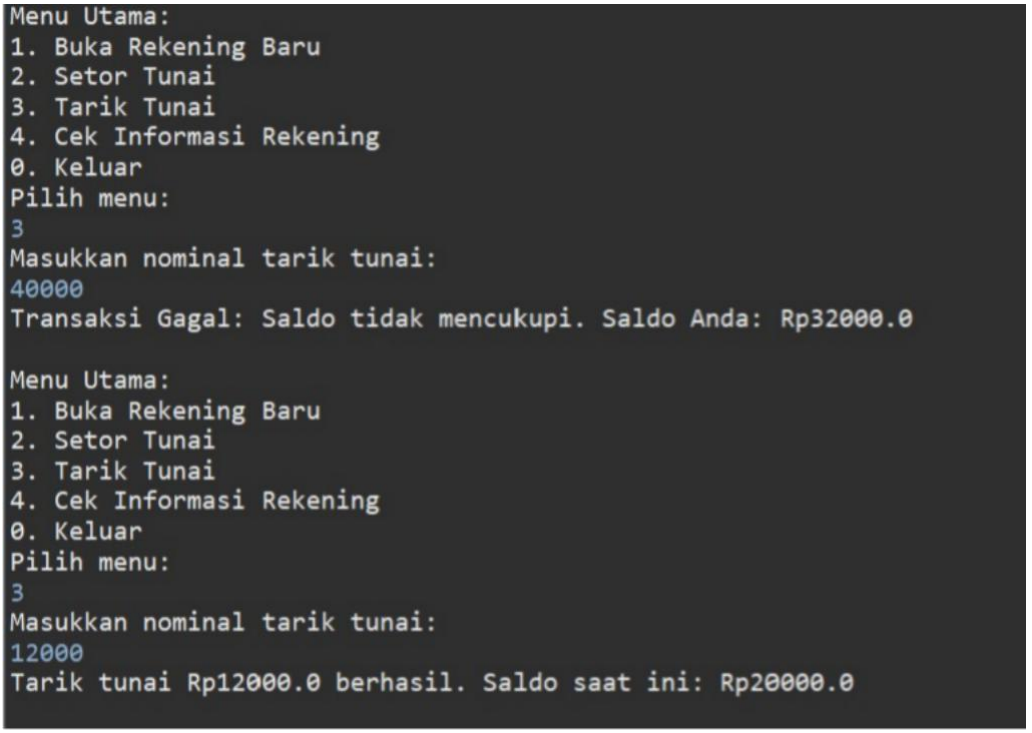
```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: mohon maaf, anda belum memiliki nomor rekening!");
    } else {
        System.out.println("Masukkan nominal tarik tunai: ");
        double tarik = input.nextDouble();
        akunAktif.tarikTunai(tarik);
    }
    break;
```

Hasil program jika dijalankan:

```
"=== SISTEM PERBANKAN MINI ==="

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
1
Masukkan No Rekening: 12345678
Masukkan Nama Pemilik: aufa
Masukkan Saldo Awal: 32000
Rekening atas nama aufaberhasil dibuat dengan saldo Rp32000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu:
3
Masukkan nominal tarik tunai:
2000
Gagal: Nominal tarik tunai harus lebih dari 10000|
```



2. Tantangan Multi-Akun

Saat ini, aplikasi Main hanya dapat menyimpan 1 akun (Rekening akunAktif = null;). Jika pengguna memilih menu "Buka Rekening Baru" untuk kedua kalinya, akun lama akan tertimpa.

- Ubah arsitektur penyimpanan di Main.java menggunakan struktur data Collection. (Gunakan ArrayList).
- Ubah logika menu "Buka Rekening": Setiap kali pengguna membuka rekening baru, objek tersebut ditambahkan ke dalam ArrayList.
- (Bonus): Tambahkan menu baru untuk "Ganti Akun", di mana sistem akan meminta nomor rekening, kemudian mencari objek yang cocok di dalam ArrayList untuk dijadikan akunAktif

Jawaban:

- (ArrayList<Rekening>): Digunakan sebagai wadah untuk menampung banyak objek rekening sekaligus secara dinamis.
- Tambah Akun Baru (Menu 1): Setiap objek rekening baru otomatis disimpan ke dalam list menggunakan .add(). Ditambahkan perulangan for-each untuk menolak jika nomor rekening yang diinput sudah ada.
- Fitur Ganti Akun (Menu 5): Menggunakan perulangan for-each untuk mencocokkan nomor rekening input pengguna dengan data di ArrayList. Jika cocok, variabel akunAktif dialihkan ke objek tersebut.
- Indikator Status: Menampilkan nama dan nomor rekening yang sedang aktif di bagian paling atas menu konsol pada setiap perulangan.

Kode Program:

```java
package Pekan1_2211531012;
import java.util.Scanner;
public class Main {
    public static void main(String[]args) {
    Scanner input   new Scanner(System.in);
    ArrayList<Rekening> daftarRekening = new ArrayList<>();
    Rekening akunAktif = null;
    boolean isRunning = true;

    System.out.println("\"=== SISTEM PERBANKAN MINT ===\"");

    while (isRunning) {
        if (akunAktif != null) {
            System.out.println("\n[ Akun Aktif: " + akunAktif.namaPemilik + " ( No Rekening: " + akunAktif.nomorRekening + ") ]");
        } else {
            System.out.println("\n[ Akun Aktif: Belum ada akun terpilih ]");
        }
        System.out.println("\nMenu Utama:");
        System.out.println("1. Buka Rekening Baru");
        System.out.println("2. Setor Tunai");
        System.out.println("3. Tarik Tunai");
        System.out.println("4. Cek Informasi Rekening");
        System.out.println("5. Ganti Akun");
        System.out.println("0. Keluar");
        System.out.print("Pilih menu:");

        int pilihan   input.nextInt();
        input.nextLine();

        switch (pilihan) {
            case 1:
            System.out.print("Masukkan No Rekening: ");
            String no = input.nextLine();
            boolean sudahAda = false;
            for (Rekening r : daftarRekening) {
                if (r.nomorRekening.equalsIgnoreCase(no)) {
                    sudahAda = true;
                    break;
                }
            }
```

```java
            if (sudahAda) {
                System.out.println("Error: Nomor rekening tersebut sudah terdaftar!");
            } else {
                System.out.print("Masukkan Nama Pemilik: ");
                String nama = input.nextLine();
                System.out.print("Masukkan Saldo Awal: ");
                Double saldo = input.nextDouble();
                Rekening rekeningBaru   new Rekening(no, nama, saldo);
                daftarRekening.add(rekeningBaru);
                akunAktif   rekeningBaru;
            }
            break;

            case 2:
                if (akunAktif == null) {
                    System.out.println("Error: mohon maaf, anda belum memiliki nomor rekening!");
                } else {
                    System.out.println("Masukkan nominal setor: ");
                    double setor = input.nextDouble();
                    akunAktif.setorTunai(setor);
                }
                break;

            case 3:
                if (akunAktif    null) {
                    System.out.println("Error: mohon maaf, anda belum memiliki nomor rekening!");
                } else {
                    System.out.println("Masukkan nominal tarik tunai: ");
                    double tarik = input.nextDouble();
                    akunAktif.tarikTunai(tarik);
                }
                break;

            case 4:
                if (akunAktif == null) {
                    System.out.println("Error: Anda belum membuka rekening!");
                } else {
                    akunAktif.cekInformasi();
                }
                break;
```

```
            case 5:
                if (daftarRekening.isEmpty()) {
                    System.out.println("Error: Belum ada rekening yang terdaftar di dalam sistem!");
                } else {
                    System.out.print("Masukkan No Rekening yang ingin diakses: ");
                    String cariNo = input.nextLine();

                    Rekening akunDitemukan = null;
                    for (Rekening r : daftarRekening) {
                        if (r.nomorRekening.equalsIgnoreCase(cariNo)) {
                            akunDitemukan = r;
                            break;
                        }
                    }

                    if (akunDitemukan != null) {
                        akunAktif = akunDitemukan;
                        System.out.println("Berhasil beralih ke rekening atas nama " + akunAktif.namaPemilik);
                    } else {
                        System.out.println("Error: Nomor rekening " + cariNo + " tidak ditemukan!");
                    }
                }
                break;

            case 0:
                isRunning = false;
                System.out.println("Sistem ditutup. Terima kasih!");
                break;

            default:
                System.out.println("Pilihan tidak valid!");

        }
    }
    input.close();
    }
}
```

Hasil:

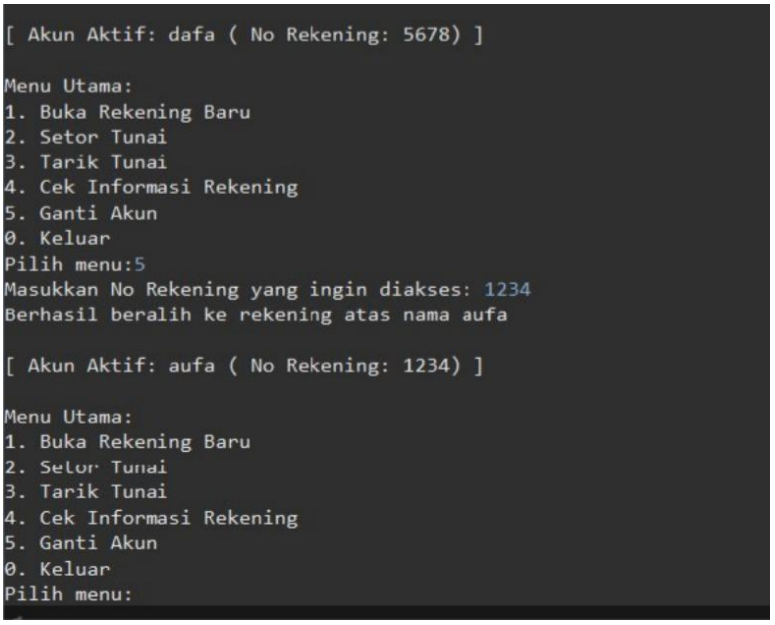
```
"=== SISTEM PERBANKAN MINI ==="

[ Akun Aktif: Belum ada akun terpilih ]

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:1
Masukkan No Rekening: 1234
Masukkan Nama Pemilik: aufa
Masukkan Saldo Awal: 1000000
Rekening atas nama aufaberhasil dibuat dengan saldo Rp1000000.0

[ Akun Aktif: aufa ( No Rekening: 1234) ]

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:1
Masukkan No Rekening: 5678
Masukkan Nama Pemilik: dafa
Masukkan Saldo Awal: 20333333
Rekening atas nama dafaberhasil dibuat dengan saldo Rp2.0333333E7
```


[Akun Aktif: dafa (No Rekening: 5678)]
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:5
Masukkan No Rekening yang ingin diakses: 1234
Berhasil beralih ke rekening atas nama aufa
[Akun Aktif: aufa (No Rekening: 1234)]
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:

LAPORAN PRATIKUM
PEMOGRAMAN BERORIENTASI OBJEK

LAPORAN PRAKTIKUM PEKAN 2

Disusun Oleh:
Muhammad Aufa Rafiki

2511531012

Dosen Pengampu: Nurfiah, S.ST, M.Kom

Asisten Praktikum: Ezza Addini



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

PEMBAHASAN

A. Langkah Kerja Praktikum

kita akan membuat objek Transaksi setiap kali terjadi aktivitas penyetoran atau penarikan tunai, lalu menyimpannya di dalam objek Rekening.

1. Membuat Class Diagram Transaksi

Langkah Kerja:

1) Buat class dengan nama Transaksi

```
public class Transaksi {
```

2) Buat attribut dan constructor pada class Transaksi

Class Transaksi memiliki 3 buat attribute yaitu idTransaksi, jenis dan nominal. Jenis transaksi ada 2 buah yaitu Debit digunakan untuk penarikan dan Kredit digunakan untuk penyetoran.

```
String idTransaksi;
String jenis;
double nominal;

//construktor
public Transaksi(String id, String jenis, double nominal) {
    this.idTransaksi = id;
    this.jenis = jenis;
    this.nominal = nominal;
}
```

Penjelasan Program:

- String idTransaksi; : Mendeklarasikan variabel idTransaksi bertipe String untuk menyimpan kode identitas unik transaksi.
- String jenis; : Mendeklarasikan variabel jenis bertipe String untuk menyimpan jenis atau kategori transaksi
- double nominal; : Mendeklarasikan variabel nominal bertipe double untuk menyimpan nilai uang transaksi.
- Construktor
  - Method khusus yang otomatis dijalankan saat Anda membuat objek baru dari kelas Transaksi
  - Meminta 3 nilai parameter awal (id, jenis, nominal) untuk langsung diisikan ke atribut kelas menggunakan kata kunci this
  - this.idTransaksi = id; : Memasukkan nilai dari variabel masukan id ke dalam atribut idTransaksi milik objek ini.

- this.jenis = jenis; : Memasukkan nilai dari variabel masukan jenis ke dalam atribut jenis milik objek ini.
- this.nominal = nominal; : Memasukkan nilai dari variabel masukan nominal ke dalam atribut nominal milik objek ini.

3) Membuat method

Kita akan membuat method cetakDetail() yang nantinya akan digunakan untuk menampilkan 1 baris detail transaksi.

```
public void cetakDetail() {
    System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
}
```

Penjelasan Program:

- Sebuah method untuk menampilkan seluruh informasi/detail data transaksi ke layar terminal dalam satu baris teks ringkas.
- Mencetak informasi gabungan dari ID, jenis, dan nominal transaksi ke layar terminal dalam satu baris.

Kode Program lengkap Class Transaksi.

```
1  package Praktikum2_2511531012_Muhammad4ufaRafiki;
2
3  public class Transaksi {
4      String idTransaksi;
5      String jenis;
6      double nominal;
7
8      //constructor
9      public Transaksi(String id, String jenis, double nominal) {
10         this.idTransaksi = id;
11         this.jenis = jenis;
12         this.nominal = nominal;
13     }
14
15     public void cetakDetail() {
16         System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
17     }
18
19 }
20
```

2. Membuat Relasi pada Rekening.java

Buka kembali file Rekening.java dari Praktikum sebelumnya. Kita akan menambahkan atribut baru yang merepresentasikan relasi has-a yaitu satu rekening memiliki banyak transaksi.

```java
import java.util.ArrayList;

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    //Implementasi asosiasi (1-to-many)
    ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        //wajib menginisialisasi ArrayList di dalam Construktor agar tidak NullPointerExcepti0n
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + "berhasil dibuat dengan saldo Rp" + saldo);
    }
```

Penjelasan Program:

- import java.util.ArrayList; : Mengimpor kelas ArrayList untuk membuat daftar transaksi yang ukurannya dapat bertambah secara dinamis.
- Deklarasi atribut à nomorRekening, namaPemilik, saldo : Atribut untuk menyimpan data dasar rekening (nomor, nama pemilik, dan jumlah saldo).
- ArrayList<Transaksi> riwayatTransaksi; : Mendeklarasikan variabel riwayatTransaksi berbentuk ArrayList untuk menampung kumpulan objek Transaksi.
- Method Construktor
  public Rekening(String nomor, String nama, double saldoAwal) {}
  - Inisialisasi Data: Mengisi nilai atribut nomorRekening, namaPemilik, dan saldo sesuai parameter yang dimasukkan saat pembuatan objek.
  - Inisialisasi Listà this.riwayatTransaksi = new ArrayList<>(); Membuat dan menginisialisasi objek ArrayList baru yang kosong ke dalam atribut riwayatTransaksi.
  - Cetak Status: Menampilkan pesan bahwa rekening baru telah berhasil dibuat beserta nama dan saldo awalnya

3. Mencatat Riwayat Transaksi

Silahkan peraiki method setorTunai dan tarikTunai di class Rekening.java agar secara otomatis menginstansiasi objek Transaksi dan menyimpannya ke dalam riwayatTransaksi saat transaksi berhasil.

```java
public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        //merekam riwayat
        String idTrx = "TRS-S-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
    } else {
        System.out.println("Gagal: Nominal setor harus lebih dari 0!");
    }
}
```

Penjelasan Program:

- method setorTunai dengan parameter nominal bertipe double untuk memproses penambahan saldo.
- if (nominal > 0) { : Memeriksa apakah nominal uang yang disetor bernilai valid (lebih besar dari 0).
  - saldo += nominal; : Menambahkan jumlah nominal penyetoran ke dalam total saldo rekening saat ini.
  - String idTrx = "TRS-S-" + System.currentTimeMillis(); : Membuat kode unik ID transaksi dengan menggabungkan awalan "TRS-S-" dan angka waktu sistem dalam milidetik.
  - Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal); : Menginstansiasi objek Transaksi baru dengan jenis "Kredit" dan memasukkan parameter idTrx serta nominal
  - riwayatTransaksi.add(trxBaru); : Menyimpan objek trxBaru ke dalam daftar riwayatTransaksi (ArrayList).
  - Mencetak pesan konfirmasi ke terminal bahwa proses setor tunai berhasil beserta saldo terbarunya.
- } else { : Menandakan kondisi alternatif jika pemeriksaan awal nominal > 0 tidak terpenuhi (nominal 0 atau negatif).
  - Mencetak pesan peringatan ke terminal jika nominal setor tidak valid.

B. Latihan /Tugas

1. Integrasi Rekam Jejak Penarikan (30%)

- Buka kembali method tarikTunai(double nominal) di dalam file Rekening.java.
- Tambahkan logika perekaman objek Transaksi seperti pada contoh setorTunai. Pastikan jenis transaksinya dicatat sebagai "Debit" dan kodenya menggunakan prefiks "TRX-T-". Transaksi hanya boleh dicatat

jika penarikan berhasil (saldo mencukupi).

Jawaban:

```
public void tarikTunai(double nominal) {
    if (nominal < 10000){
        System.out.println("Gagal: Nominal tarik tunai harus lebih dari 10000!");
    } else if (nominal > saldo) {
        System.out.println( "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
    } else {
        saldo -= nominal;

        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit ", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);

    }
}
```

Penjelasan Program:

- Mendeklarasikan method tarikTunai dengan parameter nominal bertipe double untuk memproses penarikan saldo.
- if (nominal < 10000){ : Memeriksa kondisi pertama apakah nominal penarikan kurang dari batas minimal (10.000).
  - Mencetak pesan gagal jika nominal yang dimasukkan di bawah 10.000.
- } else if (nominal > saldo) { : Memeriksa kondisi kedua apakah nominal penarikan melebihi saldo yang tersedia di rekening.
  - Mencetak pesan gagal beserta sisa saldo saat ini jika saldo tidak mencukupi.
- } else { : Blok yang dieksekusi jika kedua kondisi kegagalan di atas tidak terpenuhi (artinya penarikan berhasil).
  - saldo -= nominal; : Mengurangi nilai saldo dengan nominal penarikan.
  - String idTrx = "TRX-T-" + System.currentTimeMillis(); : Membuat kode ID unik transaksi menggunakan prefiks dan angka waktu sistem dalam milidetik.
  - Transaksi trxBaru = new Transaksi(idTrx, "Debit ", nominal); : Menginstansiasi objek Transaksi baru dengan jenis transaksi debit dan jumlah nominalnya.
  - riwayatTransaksi.add(trxBaru); : Menyimpan objek transaksi baru ke dalam daftar riwayatTransaksi (ArrayList).
  - Mencetak pesan konfirmasi penarikan berhasil beserta jumlah saldo yang tersisa.

2. **Fitur Cetak Mutasi Rekening (70%)**
   - Buat sebuah method baru di dalam class Rekening.java dengan nama cetakMutasi().
   - Di dalam method tersebut, gunakan struktur perulangan for-each untuk menelusuri seluruh data di dalam koleksi riwayatTransaksi. Panggil method cetakDetail() milik setiap objek Transaksi. Jika riwayat kosong, cetak pesan: "Belum ada transaksi pada rekening ini".
   - Buka file Main.java, tambahkan menu baru: "6. Cetak Mutasi (Riwayat)". Panggil method cetakMutasi() milik akunAktif saat pengguna memilih menu tersebut.

   Jawaban:
   - Method cetakMutasi().



```
public void cetakMutasi() {
    System.out.println("\n--- Riwayat Transaksi ---");
    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening ini");
    } else {
        for (Transaksi trx : riwayatTransaksi) {
            trx.cetakDetail();
        }
    }
}
```

   Penjelasan:
   - Mendeklarasikan method publik bernama cetakMutasi tanpa nilai kembalian (*void*).
   - Mencetak judul header "Riwayat Transaksi" diawali dengan baris baru
   - if (riwayatTransaksi.isEmpty()) { : Memeriksa apakah koleksi riwayatTransaksi dalam keadaan kosong (belum ada transaksi).
     - Mencetak pesan pemberitahuan Belum ada transaksi pada rekening ini
   - else { : Blok yang dieksekusi jika daftar riwayatTransaksi berisi minimal satu transaksi.
     - for (Transaksi trx : riwayatTransaksi) { : Perulangan *for-each* untuk mengambil setiap objek Transaksi (diberi nama variabel sementara trx) di dalam daftar riwayatTransaksi secara berurutan.
     - trx.cetakDetail(); : Memanggil method cetakDetail() milik objek trx untuk menampilkan informasi detail masing-masing transaksi ke layar.

- Tambahan Menu Baru "6. Cetak Mutasi (Riwayat)"

```
System.out.println("\nMenu Utama:");
System.out.println("1. Buka Rekening Baru");
System.out.println("2. Setor Tunai");
System.out.println("3. Tarik Tunai");
System.out.println("4. Cek Informasi Rekening");
System.out.println("5. Ganti Akun");
System.out.println("6. Cetak Mutasi (Riwayat)");
System.out.println("0. Keluar");
System.out.print("Pilih menu:");
```

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        akunAktif.cetakMutasi();
    }
    break;
```

Penjelasan Program:

- System.out.println("6. Cetak Mutasi (Riwayat)"); : Menambahkan pilihan menu ke-6 pada daftar menu utama di konsol terminal.
- Logika Pemrosesan Menu (case 6:):
    - case 6: : Pilihan menu yang akan dijalankan jika pengguna memasukkan angka 6.
    - if (akunAktif == null) { : Memeriksa apakah pengguna belum memilih atau membuka rekening aktif.
        - System.out.println("Error: Anda belum membuka rekening!"); : Mencetak pesan kesalahan jika variabel akunAktif masih bernilai null.
    - } else { : Blok yang dieksekusi jika rekening aktif sudah tersedia (akunAktif tidak null).
    - akunAktif.cetakMutasi(); : Memanggil method cetakMutasi() dari objek akunAktif untuk menampilkan seluruh riwayat transaksinya.
    - break; : Menghentikan proses percabangan switch-case agar program tidak berlanjut ke case berikutnya.

Link GitHub:
https://github.com/MuhammadAufaRafiki/PrakPBO_B_2511531012

LAPORAN PRATIKUM
PEMOGRAMAN BERORIENTASI OBJEK

LAPORAN PRAKTIKUM PEKAN 3

Disusun Oleh:
Muhammad Aufa Rafiki

2511531012

Dosen Pengampu: Nurfiah, S.ST, M.Kom

Asisten Praktikum: Ezza Addini



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

PEMBAHASAN

A. Tahapan Pelaksanaan Praktikum

Langkah-Langkah:

1. Mengunci Entitas Transaksi

   Buka file Transaksi.java. Lalu ubah variabelnya menjadi private dan memberikan Getter hanya untuk data yang perlu dibaca. Objek transaksi bersifat immutable atau tidak boleh diubah setelah dibuat, sehingga kita tidak membuat Setter. sistem membatasi akses pada entitas transaksi agar objek bersifat immutable atau tidak dapat diubah setelah dibuat.

```java
package Praktikum3_2511531012;

public class Transaksi {
    private String idTransaksi;
    private String jenis;
    private double nominal;

    //construktor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    public String getIdTransaksi() { return idTransaksi;}
    public String getJenis() { return jenis;}
    public double getNominal() { return nominal;}

    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
    }

}
```

   Penjelasan Kode:

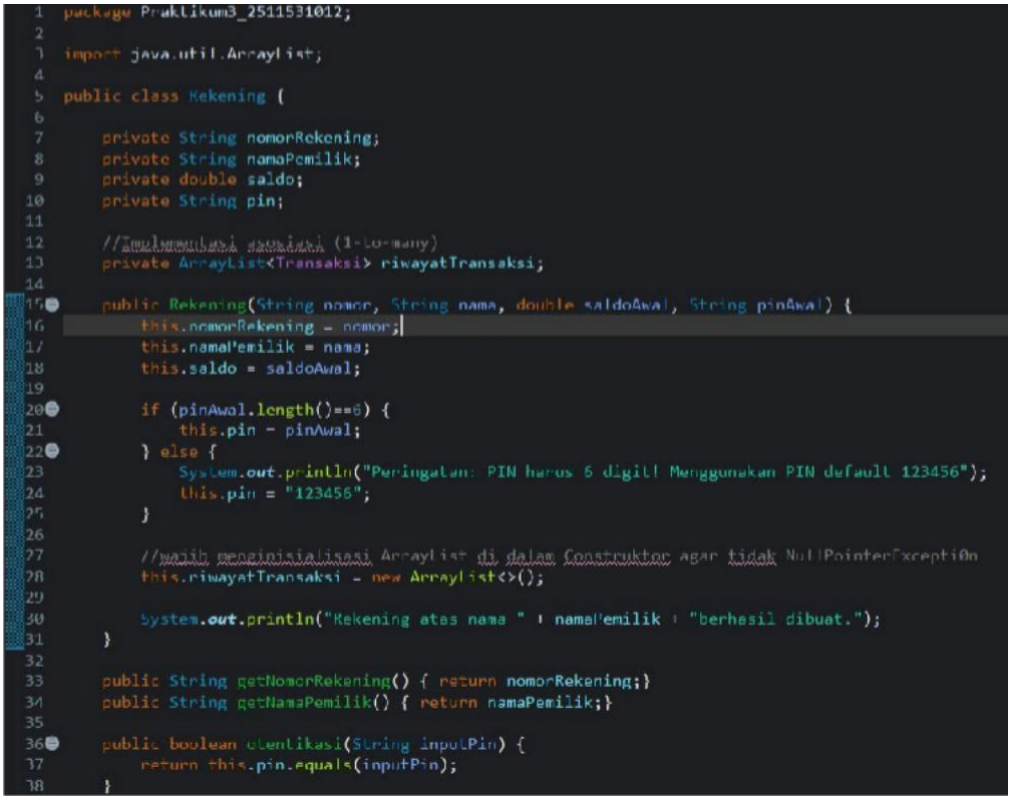
   - Atribut Kelas (Private):

     Atribut idTransaksi, jenis, dan nominal diubah menjadi private agar tidak bisa diakses atau diubah langsung dari luar *class*.
   - Konstruktor

     Transaksi(String id, String jenis, double nominal) digunakan untuk mengisi nilai atribut-atribut tersebut saat objek pertama kali diinisialisasi.
   - metode Getter (getIdTransaksi(), getJenis(), getNominal()) sehingga data bersifat read-only (hanya bisa dibaca). Tidak ada metode Setter yang dibuat.
   - Metode cetakDetail() disediakan untuk menampilkan riwayat transaksi ke layar.

2. Mengunci Entitas Rekening dan Menambahkan PIN

Buka file Rekening.java. Kita akan mengamankan saldo dan menambahkan atribut PIN.



```java
package Praktikum3_2511531012;

import java.util.ArrayList;

public class Rekening {

    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin;

    //Implementasi composition (1-to-many)
    private ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        if (pinAwal.length()==6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
            this.pin = "123456";
        }

        //wajib menginisialisasi Arraylist di dalam Constructor agar tidak NullPointerException
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + "berhasil dibuat.");
    }

    public String getNomorRekening() { return nomorRekening;}
    public String getNamaPemilik() { return namaPemilik;}

    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }
```

Penjelasan Kode:

- Seluruh atribut (nomorRekening, namaPemilik, saldo, pin, riwayatTransaksi) dikunci dengan modifier private.
- Konstruktor Rekening dimodifikasi untuk menerima parameter pinAwal dan menyertakan validasi logika. Jika panjang pinAwal adalah 6 digit, maka PIN tersebut disimpan. Jika tidak, sistem akan mencetak pesan peringatan dan otomatis menggunakan PIN default "123456".
- Metode Getter: getNomorRekening() dan getNamaPemilik() menyediakan akses read only (hanya baca) untuk mengambil data nomor rekening dan nama pemilik. Metode getPin() dan setSaldo() sengaja tidak dibuat untuk mencegah kebocoran kode rahasia serta manipulasi nilai saldo secara tidak sah dari luar kelas.
- Sistem menggunakan metode otentikasi(String inputPin) untuk mencocokkan input PIN dari pengguna dengan atribut pin yang

tersimpan. Metode ini akan mengembalikan nilai true jika cocok, dan false jika salah.

B. Latihan/Tugas

1. Menu 1 (Buka Rekening): Sebelum menginstansiasi objek baru, minta user menginputkan PIN (String angka 6 digit).

```
case 1:
System.out.print("Masukkan No Rekening: ");
String no = input.nextLine();
boolean sudahAda = false;
for (Rekening r : daftarRekening) {
    if (r.getNomorRekening().equalsIgnoreCase(no)) {
        sudahAda = true;
        break;
    }
}
if (sudahAda) {
    System.out.println("Error: Nomor rekening tersebut sudah terdaftar!");
} else {
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    Double saldo = input.nextDouble();
    input.nextLine();
    //Input PIN
    System.out.print("Masukkan PIN (6 digit): ");
    String pin = input.nextLine();

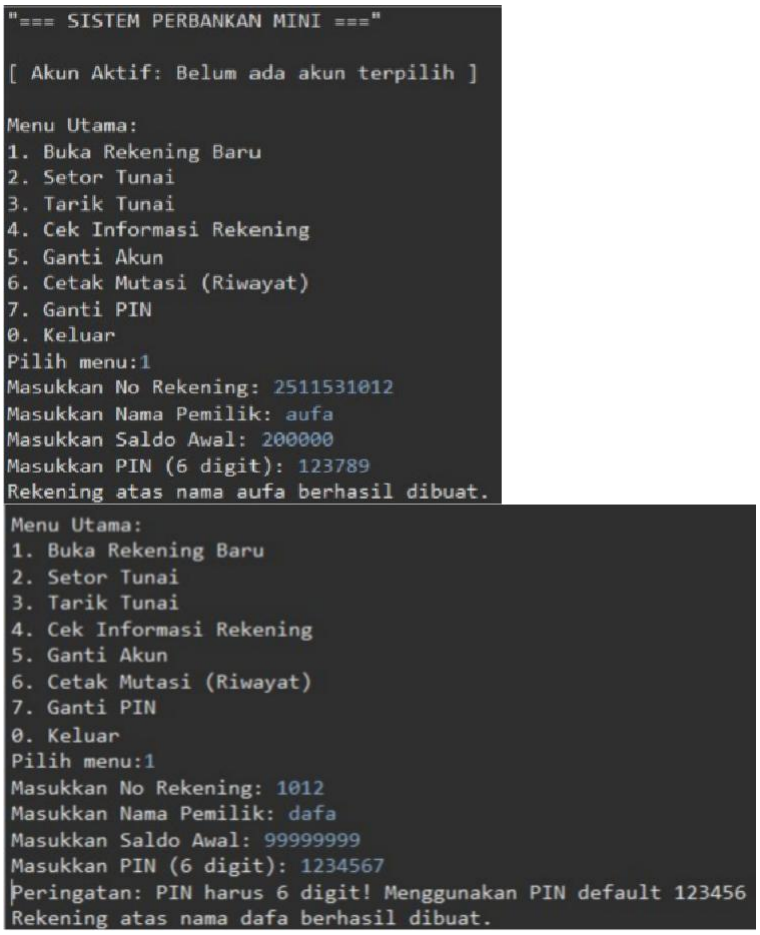
    Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
}
break;
```

Penjelasan Kode:

- Input Nomor Rekening: Program meminta pengguna memasukkan nomor rekening yang diinginkan.
- Pencegahan Duplikasi: Program melakukan perulangan (looping) untuk mengecek seluruh daftar rekening yang tersimpan. Penggunaan r.getNomorRekening() menerapkan prinsip enkapsulasi (getter). Jika nomor rekening yang diinput sudah ada di sistem, variabel sudahAda diubah menjadi true.
- Jika sudahAda bernilai true: Tampil pesan error bahwa nomor rekening tidak bisa digunakan karena sudah terdaftar.
- Jika sudahAda bernilai false: Program berlanjut ke blok else untuk memproses pembuatan rekening baru.

- Program meminta data Nama Pemilik dan Saldo Awal.
- input.nextLine() setelah nextDouble() berfungsi untuk membersihkan karakter enter (buffer) agar input berikutnya berjalan lancar.
- Program meminta pengguna memasukkan PIN 6 digit yang akan digunakan sebagai pengaman transaksi.
- Pembuatan Objek: Membuat objek rekeningBaru dari kelas Rekening dengan menyertakan parameter no, nama, saldo, dan pin ke dalam konstruktor.
- Objek baru disimpan ke dalam list (daftarRekening.add).
- Objek yang baru dibuat secara otomatis dijadikan sebagai akun yang sedang diakses saat ini (akunAktif).

Output:



Penjelasan Output:

Jika panjang pin adalah 6 digit, maka PIN tersebut disimpan. Jika tidak, sistem akan mencetak pesan peringatan dan otomatis menggunakan PIN default "123456".

2. Keamanan Transaksi: Perbarui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi). Sebelum sistem mengeksekusi kedua method tersebut, sistem harus meminta user memasukkan PIN.
   - Panggil fungsi akunAktif.otentikasi(pinYangDiinput).
   - Jika mengembalikan nilai true, jalankan transaksi.
   - Jika false, batalkan dan cetak: "Akses Ditolak: PIN yang Anda masukkan salah!".

   Jawaban:

   - Perbarui Menu 3 (Tarik Tunai)

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: mohon maaf, anda belum memiliki nomor rekening!");
    } else {
        // Minta PIN sebelum Tarik Tunai
        System.out.print("Masukkan PIN (6 digit): ");
        String inputPin = input.nextLine();

        // Otentikasi PIN
        if (akunAktif.otentikasi(inputPin)) {
            System.out.print("Masukkan nominal tarik tunai: ");
            double tarik = input.nextDouble();
            input.nextLine();
            akunAktif.tarikTunai(tarik);
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;
```

   Penjelasan Kode:

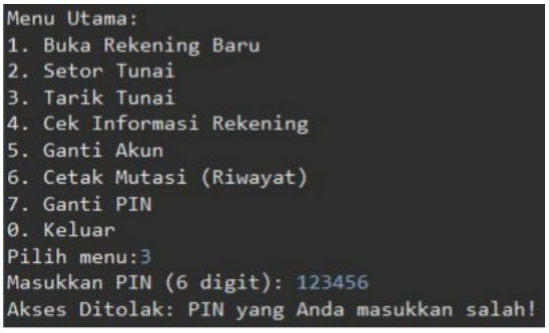
   - Program mengecek apakah variabel akunAktif bernilai null.
   - Jika null, berarti pengguna belum membuka atau memilih rekening. Program mencetak pesan error dan proses penarikan dibatalkan.
   - Jika akun aktif tersedia, program meminta pengguna menginputkan PIN terlebih dahulu sebagai verifikasi keamanan sebelum penarikan uang diproses.
   - Pemanggilan akunAktif.otentikasi(inputPin) mengirimkan PIN

input untuk dicocokkan dengan PIN rahasia yang tersimpan di dalam objek Rekening.

- Jika Otentikasi Berhasil (true): Program meminta masukan nominal uang (tarik), memanggil input.nextLine() untuk membersihkan karakter enter (buffer), lalu memanggil method akunAktif.tarikTunai(tarik) untuk memproses pengurangan saldo.
- Jika Otentikasi Gagal (false): Transaksi dibatalkan secara otomatis dan sistem menampilkan pesan "Akses Ditolak: PIN yang Anda masukkan salah!".

Output:

Jika pin salah maka mjuncul pesan akses ditolak



Jika pin benar maka tarik tunai dapat dilakukan



- Menu 6 (Cetak Mutasi).

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        // Minta PIN sebelum Cetak Mutasi
        System.out.print("Masukkan PIN (6 digit): ");
        String inputPin = input.nextLine();

        // Otentikasi PIN
        if (akunAktif.otentikasi(inputPin)) {
            akunAktif.cetakMutasi();
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;
```

Penjelasan Kode:

- Program mengecek apakah variabel akunAktif bernilai null.
- Jika null, berarti belum ada akun yang dibuat atau dipilih. Program akan mencetak pesan error dan proses pencetakan mutasi langsung dibatalkan.
- Jika akun aktif tersedia, program meminta pengguna menginputkan PIN terlebih dahulu. Hal ini dilakukan untuk menjaga privasi dan keamanan data riwayat transaksi nasabah.
- Memanggil fungsi akunAktif.otentikasi(inputPin) untuk mencocokkan input pengguna dengan PIN yang tersimpan di dalam objek Rekening tanpa mengakses variabel pin secara langsung.
- Jika Otentikasi Berhasil (true): Sistem mengeksekusi metode akunAktif.cetakMutasi() untuk menampilkan seluruh riwayat transaksi.
- Jika Otentikasi Gagal (false): Akses ditolak dan sistem mencetak pesan "Akses Ditolak: PIN yang Anda masukkan salah!".

Output:

Jika pin salah

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu:6
Masukkan PIN (6 digit): 123456
Akses Ditolak: PIN yang Anda masukkan salah!
```

Jika pin benar maka riwayat dapat ditampilkan

```
[ Akun Aktif: aufa ( No Rekening: 2511531012)  ]

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
0. Keluar
Pilih menu:6
Masukkan PIN (6 digit): 123789

--- Riwayat Transaksi ---
ID: TRX-T-1790523194384 | Jenis: Debit  | Nominal: Rp12000.0
```

LAPORAN PRATIKUM
PEMOGRAMAN BERORIENTASI OBJEK

LAPORAN PRAKTIKUM PEKAN 4

Disusun Oleh:
Muhammad Aufa Rafiki

2511531012

Dosen Pengampu: Nurfiah, S.ST, M.Kom

Asisten Praktikum: Ezza Addini



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

PEMBAHASAN

A. Tahapan Pelaksanaan Praktikum

1. Memastikan Akses Modifier di Rekening.java

   Langkah Kerja:

   - Buka file Rekening.java.
   - Ubah akses modifier variabel saldo dan riwayatTransaksi dari private menjadi protected. Hal ini bertujuan agar subclass (RekeningTabungan dan RekeningGiro) dapat mengolah nilai saldo dan mencatat transaksi secara langsung tanpa mengeksposnya ke publik.
   - Biarkan atribut nomorRekening, namaPemilik, dan pin tetap bersifat private.

```
public class Rekening {

    private String nomorRekening;
    private String namaPemilik;
    private String pin;


    protected double saldo;
    protected ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;
```

2. Membuat Subclass RekeningTabungan

   Langkah Kerja:

   - Buat file baru bernama RekeningTabungan.java yang meng-extends kelas Rekening.
   - Tambahkan atribut khusus private double sukuBunga;
   - Buat konstruktor yang memanggil konstruktor induk menggunakan perintah super(nomor, nama, saldoAwal, pinAwal) pada baris pertama, kemudian menginisialisasi sukuBunga.
   - Buat method tambahBungaAkhirBulan() untuk menghitung nominal bunga, menambahkan ke saldo, serta menginstansiasi objek Transaksi baru ke dalam riwayatTransaksi.

```java
package Praktikum4_2511531012;

public class RekeningTabungan extends Rekening {

    private double sukuBunga;

    public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {

        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {


        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;


        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);
    }

}
```

Penjelasan Program:

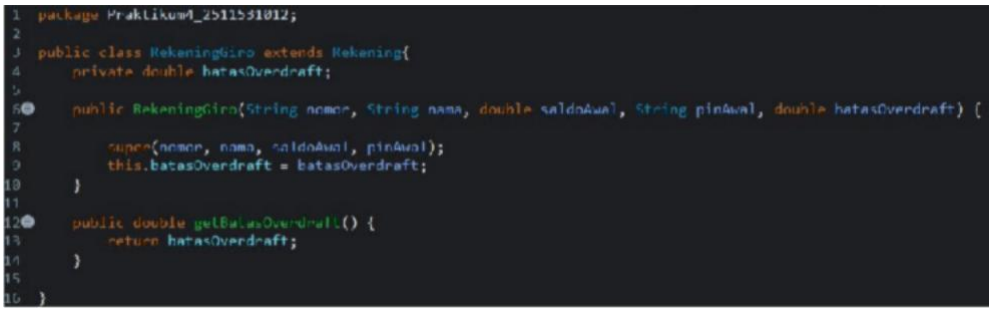
- Kelas RekeningTabungan merupakan subclass yang mewarisi seluruh sifat dan fitur dari kelas induk Rekening menggunakan kata kunci extends.
- Memiliki atribut khusus sukuBunga yang bersifat private, serta konstruktor yang memanggil super(nomor, nama, saldoAwal, pinAwal) pada baris pertama untuk menginisialisasi data induk (nomor, nama, saldoAwal, pinAwal) dan menetapkan nilai sukuBunga.
- Metode tambahBungaAkhirBulan(): Menghitung nominal bunga bulanan berdasarkan saldo, menambahkan bunga langsung ke variabel saldo (yang berakses protected), mencatat riwayat transaksi baru ke riwayatTransaksi menggunakan ID unik berbasis timestamp (System.currentTimeMillis()), dan mencetak konfirmasi ke layar.

3. Membuat Subclass RekeningGiro

   Langkah Kerja:

   - Buat file baru bernama RekeningGiro.java yang meng-extends kelas Rekening.
   - Tambahkan atribut khusus private double batasOverdraft; untuk menampung limit pinjaman nasabah bisnis.
   - Gunakan super() pada konstruktor untuk meneruskan data ke kelas induk.

- Sediakan method *getter* getBatasOverdraft().



```
package Praktikum4_2511531012;

public class RekeningGiro extends Rekening{
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {

        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    public double getBatasOverdraft() {
        return batasOverdraft;
    }

}
```

Penjelasan Program:

- Kelas RekeningGiro merupakan *subclass* yang mewarisi seluruh sifat dan atribut dari kelas induk Rekening menggunakan kata kunci extends.
- Memiliki atribut khusus batasOverdraft yang bersifat private untuk menyimpan limit pinjaman nasabah, serta konstruktor yang memanggil super(nomor, nama, saldoAwal, pinAwal) pada baris pertama untuk menginisialisasi data induk (nomor, nama, saldoAwal, pinAwal) dan menetapkan nilai batasOverdraft.
- Metode Getter (getBatasOverdraft): Menyediakan akses read-only (hanya baca) untuk mengambil nilai variabel batasOverdraft dari luar kelas sesuai prinsip enkapsulasi.

B. Latihan /Tugas

1. Pembuatan Multi-Produk

   Modifikasi case 1: Buka Rekening Baru di Main.java.

   - Setelah user menginputkan Saldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis"
   - Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.
   - Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.
   - Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

Jawaban:

```
case 1:
System.out.print("Masukkan No Rekening: ");
String no = input.nextLine();
boolean sudahAda = false;
for (Rekening r : daftarRekening) {
    if (r.getNomorRekening().equalsIgnoreCase(no)) {
        sudahAda = true;
        break;
    }
}
if (sudahAda) {
    System.out.println("Error: Nomor rekening tersebut sudah terdaftar!");
} else {
System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    Double saldo = input.nextDouble();
    input.nextLine();
    //Input PIN
    System.out.print("Masukkan PIN (6 digit): ");
    String pin = input.nextLine();

    System.out.println("Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis ");
    System.out.println("Produk yang dipilih: ");
    int pilihannya = input.nextInt();
    input.nextLine();
    switch (pilihannya) {
        case 1:
            System.out.print("Masukkan Suku Bunga (%): ");
            Double sukuBunga = input.nextDouble();

            RekeningTabungan rekeningTabungan = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
            daftarRekening.add(rekeningTabungan);
            akunAktif = rekeningTabungan;
            break;

        case 2:
            System.out.print("Masukkan Limit Pinjaman : ");
            Double  batasOverdraft = input.nextDouble();

            RekeningGiro rekeningGiro = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);
            daftarRekening.add(rekeningGiro);
            akunAktif = rekeningGiro;
            break;
    }

}
break;
```

Penjelasan Program:

- Menampilkan pilihan produk keuangan ke layar, membaca angka pilihan pengguna (pilihannya) dengan input.nextInt(), lalu mengeksekusi input.nextLine() untuk membersihkan karakter newline dari buffer input agar tidak mengganggu pembacaan berikutnya.
- case 1 (Pembuatan Rekening Tabungan): Meminta input persentase sukuBunga, lalu menginstansiasi objek baru dari kelas RekeningTabungan menggunakan parameter data nasabah yang telah diinput sebelumnya (no, nama, saldo, pin, sukuBunga).
- case 2 (Pembuatan Rekening Giro): Meminta input batasOverdraft (limit pinjaman), lalu menginstansiasi objek baru dari kelas RekeningGiro menggunakan parameter data nasabah yang telah diinput sebelumnya (no, nama, saldo, pin, batasOverdraft).

Output:

Jika produk yang dipilih 1 yaitu Tabungan umum maka rekening yang dibuat adalah rekening tabungan dengan bunga yang diinputkan.

```
"=== SISTEM PERBANKAN MINI ==="

[ Akun Aktif: Belum ada akun terpilih ]

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu:1
Masukkan No Rekening: 2511531012
Masukkan Nama Pemilik: aufa
Masukkan Saldo Awal: 999999
Masukkan PIN (6 digit): 123456
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Produk yang dipilih: 1
Masukkan Suku Bunga (%): 5
Rekening atas nama aufa berhasil dibuat.
```

Jika produk yang dipilih 2 yaitu Tabungan umum maka rekening yang dibuat adalah rekening giro dengan limit pinjaman yang diinputkan.

```
"=== SISTEM PERBANKAN MINI ==="

[ Akun Aktif: Belum ada akun terpilih ]

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu:1
Masukkan No Rekening: 2511531012
Masukkan Nama Pemilik: aufa
Masukkan Saldo Awal: 999999
Masukkan PIN (6 digit): 123456
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Produk yang dipilih: 2
Masukkan Limit Pinjaman : 50000
Rekening atas nama aufa berhasil dibuat.
```

2. Simulasi Waktu Akhir Bulan
   - Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.
   - Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof.
   - Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().
   - Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

   Jawanan:

```
System.out.println("\nMenu Utama:");
System.out.println("1. Buka Rekening Baru");
System.out.println("2. Setor Tunai");
System.out.println("3. Tarik Tunai");
System.out.println("4. Cek Informasi Rekening");
System.out.println("5. Ganti Akun");
System.out.println("6. Cetak Mutasi (Riwayat)");
System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
System.out.println("0. Keluar");
System.out.print("Pilih menu:");
```

```
case 7:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum memilih rekening aktif!");
    }
    else if (akunAktif instanceof RekeningTabungan tabungan) {
        tabungan.tambahBungaAkhirBulan();
    } else {
        System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
    }
     break;
```

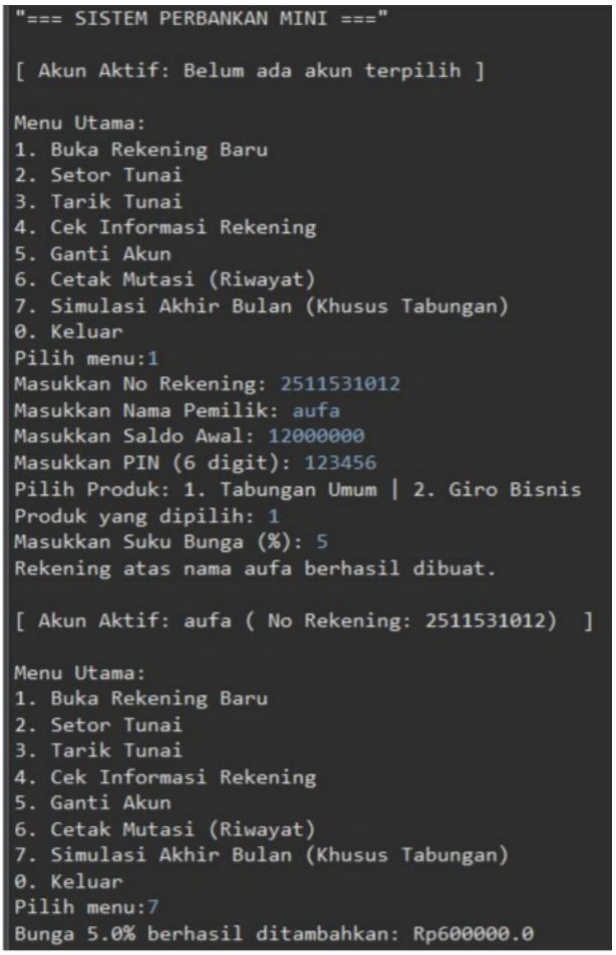
   Penjelasan Program:

- Jika pada menu utama user memilih angka 7 yaitu Simulasi Akhir Bulan (Khusus Tabungan), maka program alan memeriksa apakah akunAktif bernilai null untuk memastikan pengguna sudah memilih atau membuat rekening sebelum melakukan simulasi.
- Lalu menggunakan operator instanceof (Pattern Matching) untuk mengecek apakah akunAktif bertipe RekeningTabungan sekaligus mengonversinya (Downcasting) secara otomatis ke dalam variabel tabungan.
- Jika objek bernilai true sebagai RekeningTabungan, program akan memanggil metode tambahBungaAkhirBulan(). Jika objek bukan

RekeningTabungan (misalnya RekeningGiro), program menampilkan pesan penolakan.

Output:

Jika akun rekening yang dibuat itu rekening tabungan maka 7.Simulasi Akhir Bulan (Khusus Tabungan) dapat dijalankan



Jika akun yang dibuat bukan rekening tabungan, makan akan muncul pesan gagal


"=== SISTEM PERBANKAN MINI ==="
[Akun Aktif: Belum ada akun terpilih]
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu:1
Masukkan No Rekening: 2511531012
Masukkan Nama Pemilik: aufa
Masukkan Saldo Awal: 120000000
Masukkan PIN (6 digit): 123456
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Produk yang dipilih: 2
Masukkan Limit Pinjaman : 5000000
Rekening atas nama aufa berhasil dibuat.
[Akun Aktif: aufa (No Rekening: 2511531012)]
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu:7
Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.

LAPORAN PRAKTIKUM PEKAN 5

PEMBAHASAN

A. Tahapan Pelaksanaan Praktikum

1. Refaktor Method tarikTunai pada Kelas Induk (Rekening.java)
   Langkah Kerja:
   - Buka file Rekening.java.
   - Ubah tipe kembalian (return type) pada method tarikTunai yang semula void menjadi boolean.
   - Tambahkan logika pengkondisian:
     - Jika nominal <= 0, tampilkan pesan kesalahan dan kembalikan nilai false.
     - Jika nominal > saldo, tampilkan pesan saldo tidak mencukupi dan kembalikan nilai false.
     - Jika saldo mencukupi, kurangi saldo, catat ke riwayat transaksi, tampilkan pesan sukses, dan kembalikan nilai true.

```
61      //mengubah void menjadi boolean
62      public boolean tarikTunai(double nominal) {
63          if (nominal <=0){
64              System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
65              return false;
66          } else if (nominal > saldo) {
67              System.out.println( "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
68              return false;
69          } else {
70              saldo -= nominal;
71
72              String idTrx = "TRX-T-" + System.currentTimeMillis();
73              Transaksi trxBaru = new Transaksi(idTrx, "Debit ", nominal);
74              riwayatTransaksi.add(trxBaru);
75
76              System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
77              return true; // Transaksi sukses
78          }
79      }
```

2. Melakukan Method Overriding pada Class RekeningGiro.java
   Langkah Kerja:
   - Buka file RekeningGiro.java.
   - Tambahkan atribut/variable private double batasOverdraft beserta constructor dan getter-nya.
   - Deklarasikan method tarikTunai dengan menambahkan anotasi @Override di atasnya.
   - Implementasikan logika khusus Giro:
     - Izinkan penarikan selama nominal penarikan tidak melebihi (saldo + batasOverdraft).
     - Jika nominal melebihi batas overdraft, batalkan penarikan dan kembalikan nilai false
     - Jika memenuhi syarat, kurangi saldo (saldo dapat bernilai minus/negatif), catat transaksi "Debit Giro", dan kembalikan nilai true.

```java
package Praktikum5_2511531012;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // implementasi polimorfisme (overriding)

    public boolean tarikTunai(double nominal) {
        if (nominal <=0){
            System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
            return false;
        }
        //Logika Khusus Giro: Boleh ditarik asalkan nominal tidak melebihi (saldo + batasOverdraft)
        else if (nominal > (saldo + batasOverdraft)) {
            System.out.println("Tansaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp" + (saldo + batasOverdraft));
            return false;
        } else {
            saldo -= nominal; //saldo bisa bernilai negatif di sini (memakan overdraft)

            String idTrx = "TRX-T-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Debit ", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
            return true;
        }
    }


    public double getBatasOverdraft() { return batasOverdraft; }

}
```

3. Uji Coba Pengikatan Dinamis Jalankan Main.java.
Kita tidak perlu mengubah satu baris pun kode di Menu 3 (Tarik Tunai). Coba buat akun baru dan Pilih Produk: 2. Giro. Beri saldo 100.000 dan batas overdraft 500.000. Coba tarik tunai 400.000. Jika ini Rekening biasa, pasti ditolak. Namun karena objeknya adalah RekeningGiro, Java akan otomatis memanggil logika di Langkah 2, dan saldo akan menjadi minus 300.000. Inilah keajaiban Dynamic Dispatch.

```
[ Akun Aktif: Belum ada akun terpilih ]

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 123456
Masukkan Nama Pemilik: aufa
Masukkan Saldo Awal: 100000
Masukkan PIN (6 digit): 123456
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Produk yang dipilih: 2
Masukkan Limit Pinjaman: 500000
Rekening atas nama aufa berhasil dibuat.

[ Akun Aktif: aufa (No Rekening: 123456) ]

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 3
Masukkan PIN (6 digit): 123456
Masukkan nominal tarik tunai: 400000
Tarik tunai Rp400000.0 berhasil. Saldo saat ini: Rp-300000.0
```

B. Latihan /Tugas

Tugas: Fitur Transfer Antar Rekening

1. Buat Method Transfer : Di dalam class Rekening.java, tambahkan satu method bar dengan nama :
   public void transfer(Rekening tujuan, double nominal, String pinInput)
   - Method ini menerima parameter polimorfik Rekening tujuan, bisa menerima Tabungan atau Giro.
   - Logika Method
     - Pertama, lakukan otentikasi(pinInput). Jika false, batalkan.
     - Kedua, panggil this.tarikTunai(nominal). (overriding, jika this adalah Giro, ia otomatis menggunakan aturan overdraft).
     - Ketiga, periksa kembalian nilai tarikTunai. Jika true, panggil tujuan.setorTunai(nominal).

   Jawab:

```
80    public void transfer(Rekening tujuan, double nominal, String pinInput) {
81        // 1. Lakukan otentikasi PIN. Jika false, batalkan transaksi.
82        if (!otentikasi(pinInput)) {
83            System.out.println("Transfer Gagal: PIN yang Anda masukkan salah!");
84            return;
85        }
86        // 2 & 3. Panggil this.tarikTunai(nominal) dan periksa nilai kembaliannya (boolean)
87        boolean berhasilTarik = this.tarikTunai(nominal);
88
89        if (berhasilTarik) {
90            // Jika tarik tunai berhasil (termasuk jika pakai aturan overdraft di Giro), setor ke rekening tujuan
91            tujuan.setorTunai(nominal);
92            System.out.println("Transfer ke rekening " + tujuan.getNomorRekening() + " (" + tujuan.getNamaPemilik() + ") berhasil."
93                    + "Sisa Saldo saat ini: " + (this.saldo));
94        } else {
95            System.out.println("Transfer Gagal.");
96        }
97    }
```

   Penjelasan Program:
   - Deklarasi Method transfer: Method ini bertipe void dan menerima tiga parameter, yaitu Rekening tujuan (objek penerima yang bersifat polimorfik, bisa berupa Tabungan atau Giro), double nominal (jumlah uang yang ditransfer), dan String pinInput (PIN untuk verifikasi keamanan).
   - Otentikasi PIN:
     - Program melakukan pengecekan keamanan menggunakan if (!otentikasi(pinInput)).
     - Jika PIN yang dimasukkan salah (false), program akan mencetak pesan "Transfer Gagal: PIN yang Anda masukkan salah!" dan langsung menghentikan proses menggunakan perintah return.
   - Proses Penarikan Saldo Pengirim
     - Program memanggil this.tarikTunai(nominal) dan hasilnya disimpan ke dalam variabel boolean berhasilTarik.
     - Berkat konsep polimorfisme dan Dynamic Method Dispatch, jika akun pengirim (this) adalah RekeningGiro, Java akan secara otomatis menjalankan logika penarikan khusus overdraft, dan jika berupa rekening biasa, akan menggunakan aturan saldo normal.
   - Pengecekan dan Penyetoran ke Rekening Tujuan
     - Jika berhasilTarik bernilai true, program akan melanjutkan proses dengan memanggil tujuan.setorTunai(nominal) untuk menambahkan saldo ke rekening penerima.

- Program kemudian mencetak informasi detail transfer yang meliputi nomor rekening tujuan, nama pemilik, serta sisa saldo pengirim saat ini.
- Jika proses penarikan gagal (misalnya saldo tidak mencukupi atau melebihi batas overdraft), program akan masuk ke blok else dan mencetak pesan "Transfer Gagal.".

2. Integrasi ke CLI Utama: Buka Main.java dan tambahkan menu 8. Transfer ke Rekening Lain
   - Minta pengguna memasukkan Nomor Rekening Tujuan.
   - Cari nomor tersebut di dalam daftarRekening.
   - Jika ketemu, minta nominal transfer dan PIN, lalu panggil akunAktif.transfer(akunTujuan, nominal, pin).

Jawab:

```
29              System.out.println("8. Transfer ke Rekening Lain");
181         // MENU 8: TRANSFER KE REKENING LAIN
182         case 8:
183             if (akunAktif == null) {
184                 System.out.println("Error: Anda belum memilih rekening aktif sebagai pengirim!");
185             } else {
186                 System.out.print("Masukkan No Rekening Tujuan: ");
187                 String noTujuan = input.nextLine();
188
189                 // Validasi agar tidak transfer ke rekening sendiri
190                 if (noTujuan.equalsIgnoreCase(akunAktif.getNomorRekening())) {
191                     System.out.println("Error: Tidak dapat melakukan transfer ke rekening sendiri!");
192                     break;
193                 }
194                 // Cari nomor rekening tujuan di dalam daftarRekening
195                 Rekening akunTujuan = null;
196                 for (Rekening r : daftarRekening) {
197                     if (r.getNomorRekening().equalsIgnoreCase(noTujuan)) {
198                         akunTujuan = r;
199                         break;
200                     }
201                 }
202                 // Jika rekening tujuan ditemukan
203                 if (akunTujuan != null) {
204                     System.out.print("Masukkan nominal transfer: ");
205                     double nominalTransfer = input.nextDouble();
206                     input.nextLine(); // Membersihkan buffer newline
207
208                     System.out.print("Masukkan PIN Anda (6 digit): ");
209                     String pinInput = input.nextLine();
210
211                     // Panggil method transfer yang ada di class Rekening
212                     akunAktif.transfer(akunTujuan, nominalTransfer, pinInput);
213                 } else {
214                     System.out.println("Error: Nomor rekening tujuan " + noTujuan + " tidak ditemukan di dalam sistem!");
215                 }
216             }
217             break;
218
```

Pemjelasan Program:

- Pemeriksaan Akun Aktif
  - Program mengecek apakah akunAktif bernilai null.
  - Jika pengguna belum memilih rekening aktif sebagai pengirim, sistem akan menampilkan pesan eror ("Error: Anda belum memilih rekening aktif sebagai pengirim!").
- Input dan Validasi Rekening Tujuan
  - Jika akun aktif tersedia, program meminta pengguna untuk memasukkan nomor rekening tujuan (noTujuan).
  - Dilakukan validasi menggunakan equalsIgnoreCase untuk memastikan pengguna tidak melakukan transfer ke nomor rekening miliknya sendiri. Jika nomor rekening tujuan sama dengan rekening aktif, transaksi dibatalkan dengan perintah break.

- Pencarian Rekening dalam Sistem
  - Program melakukan perulangan (looping) pada daftar seluruh rekening (daftarRekening) untuk mencari data rekening yang cocok dengan noTujuan.
  - Jika ditemukan, objek rekening tersebut disimpan ke dalam variabel akunTujuan.
- Input Nominal, PIN, dan Eksekusi Transfer
  - Jika rekening tujuan ditemukan (akunTujuan != null), program meminta masukan berupa jumlah uang yang ingin ditransfer (nominalTransfer) serta PIN akun pengirim.
  - Buffer masukan dibersihkan menggunakan input.nextLine(), lalu program memanggil method polimorfik akunAktif.transfer(akunTujuan, nominalTransfer, pinInput).
- Penanganan jika Rekening Tidak Ditemukan
  - Jika setelah perulangan akunTujuan tetap bernilai null, program mencetak pesan eror bahwa nomor rekening tujuan tidak ditemukan di dalam sistem.

Output:

```
[ Akun Aktif: dafa (No Rekening: 234567) ]

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0. Keluar
Pilih menu: 8
Masukkan No Rekening Tujuan: 123456
Masukkan nominal transfer: 90000
Masukkan PIN Anda (6 digit): 234567
Tarik tunai Rp90000.0 berhasil. Saldo saat ini: Rp10000.0
Setor tunai Rp90000.0 berhasil. Saldo saat ini: Rp190000.0
Transfer ke rekening 123456 (aufa) berhasil.Sisa Saldo saat ini: 10000.0
```